In [ ]:

```
import os  # Importing the os module for file handling
import csv  # Importing the csv module to handle CSV files
from datetime import datetime, timedelta  # Importing datetime and timede

# UserAccount Class to manage user data and borrowed books
class UserAccount:
    def __init__(self, user_id, name):  # Constructor to initialize user
        self.user_id = user_id  # Setting the user ID
        self.name = name  # Setting the user name
        self.borrowed_books = []  # List to store borrowed books

    def borrow_book(self, book_title):  # Method to add a book to the bor
        self.borrowed_books.append(book_title)  # Adding the book title t

    def return_book(self, book_title):  # Method to remove a book from th
        if book_title in self.borrowed_books:  # Check if the book is in
            self.borrowed_books.remove(book_title)  # Remove the book fro
        else:
            raise ValueError("Book not found in borrowed list.")  # Raise

# Librarian class inherits from UserAccount and manages book additions an
class Librarian(UserAccount):
    def __init__(self, user_id, name):  # Constructor to iitialize librar
        super().__init__(user_id, name)  # Call the parent cnonstructor

    def add_book(self, book_manager, book_title):  # Method to add a new
        book_manager.books[book_title] = 'available'  # Add the book with
        book_manager.save_books()  # Save the updated books list to the f

    def remove_book(self, book_manager, book_title):  # Method to remove
        if book_title in book_manager.books:  # Check if the book exists
            del book_manager.books[book_title]  # Delete the book from th
            book_manager.save_books()  # Save the updated books list to t
        else:
            raise ValueError("Book not found in the system.")  # Raise an

# ManageBookLending class handles book lending and file operations
class ManageBookLending:
    def __init__(self, file_path='books.csv'):  # Constructor to initiali
        self.file_path = file_path  # Set the file path to load and save
        self.books = self.load_books()  # Load books from the file into a

    def load_books(self):  # Method to load books from the CSV file
        books = {}  # Initialize an empty dictionary to store books
        if os.path.exists(self.file_path):  # Check if the file exists
            with open(self.file_path, mode='r') as file:  # Open the file
                reader = csv.reader(file)  # Create a CSV reader object
                for row in reader:  # Loop through each row in the file
                    books[row[0]] = row[1]  # Add the book title and stat
        return books  # Return the dictionary of books

    def save_books(self):  # Method to save the books dictionary to the C
        with open(self.file_path, mode='w', newline='') as file:  # Open
```

```
            writer = csv.writer(file)  # Create a CSV writer object
            for title, status in self.books.items():  # Loop through each
                writer.writerow([title, status])  # Write the book title

    def lend_book(self, user, book_title):  # Method to lend a book to a
        if book_title in self.books and self.books[book_title] == 'availa
            self.books[book_title] = 'borrowed'  # Change the status of t
            user.borrow_book(book_title)  # Add the book to the user's bo
            self.save_books()  # Save the updated books list to the file
        else:
            raise ValueError("Book is not available.")  # Raise an error

# ReturnsAndOverduePenalties class handles book returns and calculates pe
class ReturnsAndOverduePenalties:
    def __init__(self):  # Constructor to initialize return records
        self.return_records = {}  # Dictionary to store return records

    def return_book(self, user, book_title, borrow_date):  # Method to ha
        try:
            user.return_book(book_title)  # Try to return the book
            return_date = datetime.now()  # Get the current date as the r
            self.return_records[user.user_id] = return_date  # Store the
            overdue_penalty = self.calculate_penalty(borrow_date)  # Calc
            if overdue_penalty > 0:  # Check if there is any overdue pena
                print(f"Overdue penalty for '{book_title}': ${overdue_pen
            else:
                print(f"Book '{book_title}' returned successfully.")  # D
        except ValueError as e:
            print(f"Error: {e}")  # Show error if book not found in borro

    def calculate_penalty(self, borrow_date):  # Method to calculate over
        due_date = borrow_date + timedelta(days=14)  # Books are due afte
        overdue_days = (datetime.now() - due_date).days  # Calculate the
        penalty_rate = 1  # Define the penalty rate per overdue day
        return max(0, overdue_days * penalty_rate)  # Return the penalty

# Main function to interact with the user through the console
def main():
    # Initialize user, librarian, and book manager
    user = UserAccount("U001", "Alice")  # Create a user account for Alic
    librarian = Librarian("L001", "Admin")  # Create a librarian account
    book_manager = ManageBookLending()  # Create a book manager object to
    return_manager = ReturnsAndOverduePenalties()  # Create a return mana

    while True:  # Start an infinite loop to display the menu
        # Display menu for user interaction
        print("\n****Library Management System****")  # Display system ti
        print("1. View Available Books")  # Option to view available book
        print("2. Borrow Book")  # Option to borrow a book
        print("3. Return Book")  # Option to return a book
        print("4. View Borrowed Books")  # Option to view borrowed books
        print("5. Add Book (Librarian)")  # Option for librarian to add a
        print("6. Remove Book (Librarian)")  # Option for librarian to re
        print("7. Exit")  # Option to exit the program
```

```
        choice = input("Enter your choice: ")  # Prompt user to enter the

        if choice == '1':  # If user chooses to view available books
            # View available books
            books = "\n".join([f"{title} ({status})" for title, status in
            print(f"\nAvailable Books:\n{books if books else 'No books av

        elif choice == '2':  # If user chooses to borrow a book
            # Borrow a book
            book_title = input("Enter the book title to borrow: ")  # Pro
            try:
                book_manager.lend_book(user, book_title)  # Attempt to le
                print(f"You borrowed '{book_title}'")  # Display success
            except ValueError as e:
                print(f"Error: {e}")  # Display error message if book is

        elif choice == '3':  # If user chooses to return a book
            # Return a book
            book_title = input("Enter the book title to return: ")  # Pro
            borrow_date = datetime.now()  # Set the borrow date to the cu
            return_manager.return_book(user, book_title, borrow_date)  #

        elif choice == '4':  # If user chooses to view borrowed books
            # View borrowed books
            books = "\n".join(user.borrowed_books)  # Create a string of
            print(f"\nBorrowed Books:\n{books if books else 'No books bor

        elif choice == '5':  # If librarian chooses to add a book
            # Add a book (Librarian)
            book_title = input("Enter the book title to add: ")  # Prompt
            librarian.add_book(book_manager, book_title)  # Call the meth
            print(f"Book '{book_title}' added")  # Display success messag

        elif choice == '6':  # If librarian chooses to remove a book
            # Remove a book (Librarian)
            book_title = input("Enter the book title to remove: ")  # Pro
            try:
                librarian.remove_book(book_manager, book_title)  # Attemp
                print(f"Book '{book_title}' removed")  # Display success
            except ValueError as e:
                print(f"Error: {e}")  # Display error message if book is

        elif choice == '7':  # If user chooses to exit
            # Exit the program
            print("Exiting the Library Management System.")  # Display ex
            break  # Break the loop and exit the program

        else:  # If the user enters an invalid option
            print("Invalid choice. Please try again.")  # Display an erro

# Run the main function
if __name__ == "__main__":  # If this script is run directly
    main()  # Call the main function to start the program
```

```
****Library Management System****
1. View Available Books
2. Borrow Book
3. Return Book
4. View Borrowed Books
5. Add Book (Librarian)
6. Remove Book (Librarian)
7. Exit
```

In [ ]: